

Smart Email Reply Generator (College Project)

यह डॉक्यूमेंट "Smart Email Reply Generator" प्रोजेक्ट के लिए सभी आवश्यक कोड स्निपेट्स का सारांश (summary) प्रस्तुत करता है।

नोट: Spring Boot एप्लिकेशन को चलाने से पहले, आपको `application.properties` में `gemini.api.key` को अपने वास्तविक Gemini API कुंजी से बदलना होगा।

1. Backend: Configuration File

(File: `src/main/resources/application.properties`)

महत्वपूर्ण सुधार (CRITICAL FIX): Gemini API URL को ठीक किया गया है ताकि Spring Boot सीधे Gemini API से जुड़ सके।

```
# Server Configuration
server.port=8080

# Gemini AI API Configuration
# CRITICAL FIX: The URL is now correct.
# Replace YOUR_GEMINI_API_KEY_HERE with your actual key.
gemini.api.url=[https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flas
gemini.api.key=YOUR_GEMINI_API_KEY_HERE
```

2. Backend: Main Application

(File: `src/main/java/com/emailwriter/EmailWriterApplication.java`)

```
package com.emailwriter;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Main entry point for the Spring Boot application.
 */
@SpringBootApplication
public class EmailWriterApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmailWriterApplication.class, args);
    }

}
```

3. Backend: Data Transfer Object (DTO)

(File: `src/main/java/com/emailwriter/EmailRequest.java`)

```
package com.emailwriter;
```

```

import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

/**
 * Data Transfer Object (DTO) for the incoming email generation request.
 * Maps the JSON body from the client request.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
public class EmailRequest {
    // Original email content sent by the user for which a reply is needed.
    private String emailContent;

    // The desired tone for the AI-generated reply (e.g., "professional", "friendly").
    private String tone;
}

```

4. Backend: Controller

(File: *src/main/java/com/emailwriter/EmailGeneratorController.java*)

```

package com.emailwriter;

import lombok.AllArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/**
 * REST Controller to expose the email generation endpoint.
 * Handles incoming HTTP requests and delegates to the service layer.
 */
@RestController
@RequestMapping("/api/email")
@CrossOrigin(origins = "*") // Allows cross-origin requests from any client (like React)
@AllArgsConstructor // Injects the service via constructor (recommended practice)
public class EmailGeneratorController {

    private final EmailGeneratorService emailGeneratorService;

    /**
     * POST endpoint to generate an email reply.
     * Request body should contain JSON mapping to EmailRequest DTO.
     * @param request The request DTO with email content and tone.
     * @return HTTP 200 OK with the generated reply text as the response body.
     */
    @PostMapping("/generate")
    public ResponseEntity<String> generateEmail(@RequestBody EmailRequest request) {
        try {
            String reply = emailGeneratorService.generateEmailReply(request);
            return ResponseEntity.ok(reply);
        } catch (Exception e) {
            System.err.println("Request processing failed: " + e.getMessage());
            return ResponseEntity.internalServerError().body("Failed to generate reply");
        }
    }
}

```

5. Backend: Service (Core Logic)

(File: src/main/java/com/emailwriter/EmailGeneratorService.java)

```
package com.emailwriter;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;
import org.springframework.web.reactive.function.client.WebClientResponseException;

import java.util.Collections;
import java.util.Map;

/**
 * Service class responsible for handling the core business logic,
 * primarily interacting with the Gemini AI API to generate email replies.
 */
@Service
public class EmailGeneratorService {

    // --- Gemini API Constants ---
    private static final String JSON_CONTENTS = "contents";
    private static final String JSON_PARTS = "parts";
    private static final String JSON_TEXT = "text";
    private static final String JSON_CANDIDATES = "candidates";
    private static final String SYSTEM_INSTRUCTION = "आप एक अत्यंत पेशेवर (highly professi

    private final WebClient webClient;

    // Injected from application.properties or environment variables
    @Value("${gemini.api.url}")
    private String geminiApiUrl;

    @Value("${gemini.api.key}")
    private String geminiApiKey;

    // WebClient is injected via its Builder for reactive HTTP requests
    public EmailGeneratorService(WebClient.Builder webClientBuilder) {
        this.webClient = webClientBuilder.baseUrl("").build();
    }

    /**
     * Generates an AI-powered email reply based on the content and tone.
     * @param request The incoming request containing email content and tone.
     * @return The AI-generated reply text.
     * @throws RuntimeException if the API call fails or returns an error.
     */
    public String generateEmailReply(EmailRequest request) {
        // 1. Build the User Prompt
        String userPrompt = buildUserPrompt(request);

        // 2. Craft the Gemini API request body (complex JSON structure with System Ins
        Map<String, Object> payload = Map.of(
            JSON_CONTENTS, Collections.singletonList(
                Map.of(JSON_PARTS, Collections.singletonList(
                    Map.of(JSON_TEXT, userPrompt)
                ))
            ),
            "systemInstruction", Map.of(
                JSON_PARTS, Collections.singletonList(
```

```

        Map.of(JSON_TEXT, SYSTEM_INSTRUCTION)
    )
    );

String response = null;
try {
    // 3. Execute the API call
    response = webClient.post()
        .uri(geminiApiUrl + "?key=" + geminiApiKey)
        .header("Content-Type", "application/json")
        .bodyValue(payload)
        .retrieve()
        .bodyToMono(String.class)
        .block(); // Blocking call (use block() carefully in production)
} catch (WebClientResponseException e) {
    System.err.println("API Call failed with status code: " + e.getStatusCode())
    throw new RuntimeException("External API call failed. Status: " + e.getStat
} catch (Exception e) {
    System.err.println("Network or unexpected error during API call: " + e.getM
    throw new RuntimeException("Network or unexpected error occurred.");
}

// 4. Extract and return the final reply text
if (response != null) {
    return extractResponseContent(response);
} else {
    return "Error: AI ने कोई उत्तर नहीं दिया।";
}
}

/**
 * Constructs the specific user prompt for the Gemini model, emphasizing the tone.
 */
private String buildUserPrompt(EmailRequest request) {
    StringBuilder prompt = new StringBuilder();
    prompt.append("मूल ईमेल का उत्तर तैयार करें। कृपया **").append(request.getTone()).appen
    prompt.append("Original Email:\n").append(request.getEmailContent());
    return prompt.toString();
}

/**
 * Parses the complex JSON response from the Gemini API to find the generated text.
 */
private String extractResponseContent(String response) {
    try {
        ObjectMapper mapper = new ObjectMapper();
        JsonNode rootNode = mapper.readTree(response);

        // Navigate the JSON path: candidates[0].content.parts[0].text
        return rootNode.path(JSON_CANDIDATES).get(0)
            .path(JSON_CONTENTS).path(JSON_PARTS).get(0)
            .path(JSON_TEXT).asText();

    } catch (Exception e) {
        System.err.println("AI प्रतिक्रिया को पार्स करने में त्रुटि: " + e.getMessage());
        return "त्रुटि: AI प्रतिक्रिया को संसाधित (process) नहीं कर सका।";
    }
}
}

```

6. Frontend: React UI

(File: EmailReplyGenerator.jsx)

```
import React, { useState, useCallback } from 'react';

// === CONFIGURATION ===
// The URL for the Spring Boot backend API endpoint.
// This assumes the Spring Boot app is running on port 8080.
const BACKEND_API_URL = "http://localhost:8080/api/email/generate";

// Tone options for the user
const TONES = [
  { value: "professional", label: "Professional" },
  { value: "friendly", label: "Friendly" },
  { value: "casual", label: "Casual" },
  { value: "sarcastic", label: "Sarcastic (Use with caution)" },
];

// Main application component
const App = () => {
  const [emailContent, setEmailContent] = useState('');
  const [tone, setTone] = useState(TONES[0].value);
  const [generatedReply, setGeneratedReply] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);

  // Example placeholder content to demonstrate functionality
  React.useEffect(() => {
    setEmailContent("Hi team, I noticed the Q3 marketing report is due next Friday. Cou
  }, []);

  // Function to copy the generated text to the clipboard
  const copyToClipboard = useCallback(() => {
    if (generatedReply) {
      // Use document.execCommand('copy') as navigator.clipboard might fail in an ifram
      const el = document.createElement('textarea');
      el.value = generatedReply;
      document.body.appendChild(el);
      el.select();
      document.execCommand('copy');
      document.body.removeChild(el);
      setError("Reply copied to clipboard!"); // Use error state temporarily for confir
      setTimeout(() => setError(null), 3000);
    }
  }, [generatedReply]);

  // Function to handle the API call to the Spring Boot backend
  const generateReply = useCallback(async () => {
    setIsLoading(true);
    setError(null);
    setGeneratedReply('');

    if (!emailContent.trim()) {
      setError("Please enter the email content before generating a reply.");
      setIsLoading(false);
      return;
    }

    try {
      // The payload structure matches the EmailRequest DTO in the Spring Boot backend
      const payload = {
        emailContent: emailContent,
```

```

        tone: tone,
    };

    const response = await fetch(BACKEND_API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload),
    });

    if (!response.ok) {
        // Log the error response body for debugging
        const errorText = await response.text();
        console.error("Backend Error Response:", errorText);
        throw new Error(`API Request Failed: ${response.status} - ${errorText.substring(
    }

    // The backend returns the raw generated text as a string
    const replyText = await response.text();

    if (replyText) {
        setGeneratedReply(replyText);
    } else {
        throw new Error("Received an empty reply from the backend.");
    }
} catch (err) {
    console.error("Reply Generation Error:", err);
    setError("Failed to generate reply. Ensure the Spring Boot backend is running at
} finally {
    setIsLoading(false);
}
}, [emailContent, tone]);

return (
<div className="min-h-screen bg-gray-50 p-4 md:p-8">
  <script src="https://cdn.tailwindcss.com"(https://cdn.tailwindcss.com)"></script>
  <style>{`
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&c
    body { font-family: 'Inter', sans-serif; }
    .input-area, .output-area {
      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -2px rgba(0, 0, 0, 0
    }
  `}</style>
  <div className="max-w-4xl mx-auto">
    <h1 className="text-3xl font-bold text-center text-gray-800 mb-6 border-b pb-2">
      Smart Email Reply Generator (Backend Powered)
    </h1>

    {/* 1. Input Area */}
    <div className="bg-white p-6 rounded-xl input-area mb-8">
      <label htmlFor="email-content" className="block text-sm font-semibold text-gr
        Original Email Content
      </label>
      <textarea
        id="email-content"
        rows="8"
        className="w-full p-3 border border-gray-300 rounded-lg focus:ring-sky-500
        placeholder="Paste the email you received here..."
        value={emailContent}
        onChange={(e) => setEmailContent(e.target.value)}
        disabled={isLoading}
      ></textarea>

      <div className="flex flex-col sm:flex-row items-center justify-between mt-4 s
        <div className="w-full sm:w-1/3">

```

```

<label htmlFor="tone-select" className="block text-sm font-medium text-gr
  Desired Tone
</label>
<select
  id="tone-select"
  className="w-full p-2 border border-gray-300 bg-white rounded-lg shadow
  value={tone}
  onChange={(e) => setTone(e.target.value)}
  disabled={isLoading}
>
  {TONES.map((t) => (
    <option key={t.value} value={t.value}>
      {t.label}
    </option>
  ))}
</select>
</div>

<button
  onClick={generateReply}
  disabled={isLoading || !emailContent.trim()}
  className={`w-full sm:w-2/3 h-11 flex items-center justify-center font-se
    ${isLoading || !emailContent.trim()
      ? 'bg-gray-400 cursor-not-allowed'
      : 'bg-sky-600 hover:bg-sky-700 text-white transform hover:scale-[1.01
    ]`}
  >
  {isLoading ? (
    <div className="flex items-center">
      <svg className="animate-spin -ml-1 mr-3 h-5 w-5 text-white" xmlns="http://www.w3.org/2000/svg">
        <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" strokeWidth="4">
          <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018 0z">
        </svg>
        Generating Reply...
      </div>
    ) : (
      'Generate AI Reply'
    )}
  </button>
</div>
</div>

{/* Error/Message Display */}
{error && (
  <div className="p-4 mb-4 text-center bg-red-100 text-red-700 rounded-lg trans
    {error}
  </div>
)}

{/* 2. Output Area */}
{generatedReply && (
  <div className="bg-white p-6 rounded-xl output-area mt-4">
    <h2 className="text-xl font-bold text-gray-800 mb-4 flex items-center">
      Generated Reply
      <svg xmlns="http://www.w3.org/2000/svg" (http://www.w3.org/2000/svg)" clc
        <path d="M12 2a10 10 0 0 0 0 0 20 10 10 0 0 0 0-20z">
        <path d="M12 6.5v9">
        <path d="M10 8 12 2 2 2">
        <path d="M10 16 12 2 2-2">
      </svg>
    </h2>

    <textarea
      readOnly

```

```
        rows="8"
        className="w-full p-3 border border-sky-400 bg-sky-50 text-gray-800 font-
        value={generatedReply}
      ></textarea>

      <button
        onClick={copyToClipboard}
        className="mt-4 w-full sm:w-auto px-6 py-2 bg-emerald-600 hover:bg-emeral
      >
        <svg xmlns="http://www.w3.org/2000/svg"(http://www.w3.org/2000/svg)" clc
          <rect x="9" y="9" width="13" height="13" rx="2" ry="2"></rect>
          <path d="M5 15H4a2 2 0 0 1-2-2V4a2 2 0 0 1 2-2h9a2 2 0 0 1 2 2v1"></pat
        </svg>
        Copy Reply to Clipboard
      </button>
    </div>
  )}
</div>
</div>
);
};

export default App;
```